Ana Camargo (ID 957521848)

Homework #1

Chapter 3 – Problem 9 (a,b,c,d)

a) Scatterplot matrix

```
library(ISLR2)
Auto <- read.table("Auto.data" , header=TRUE)
View(Auto)
pairs(~. , data=Auto , main="Scatterplot matrix for Auto")
```

```
> library(ISLR2)
> library(corrplot)
> Auto <- read.table("Auto.data" , header=TRUE)
> View(Auto)
> pairs(~. , data=Auto , main="Scatterplot matrix for Auto")
```

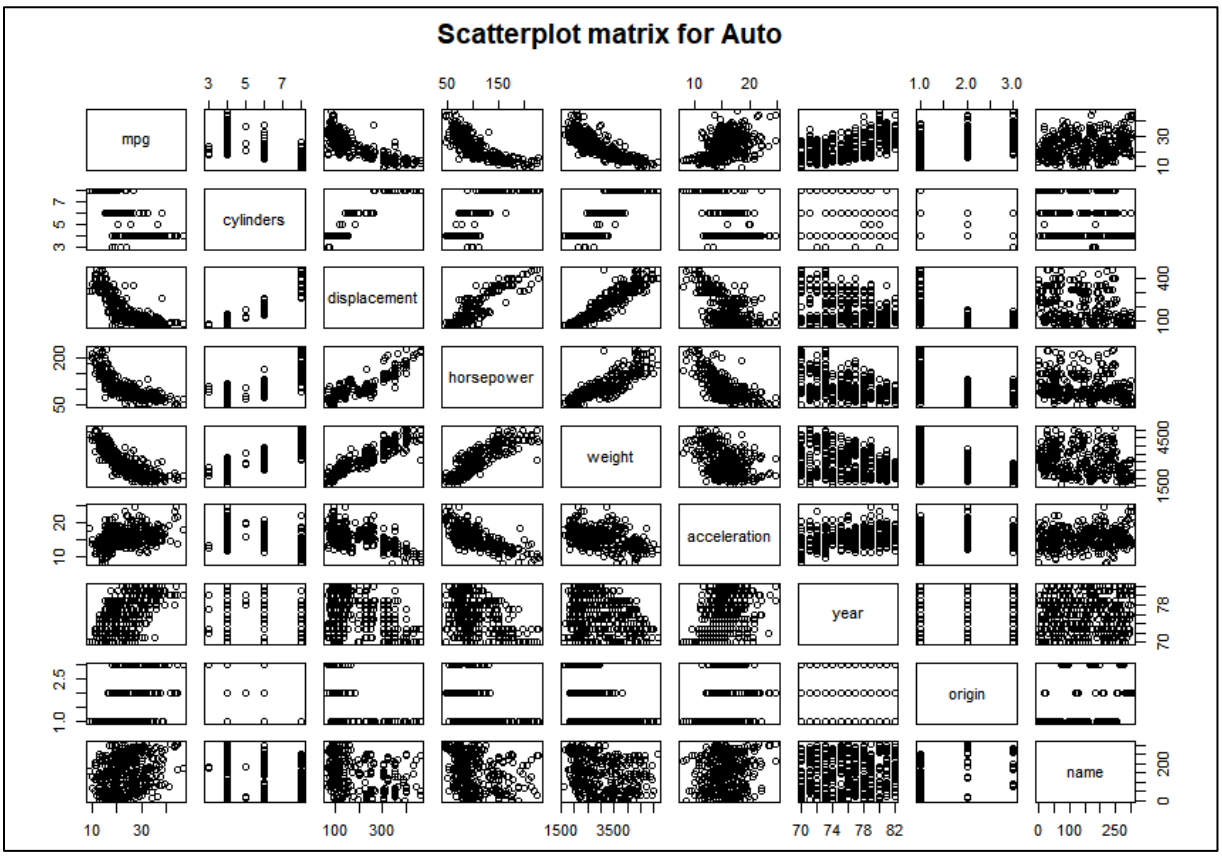


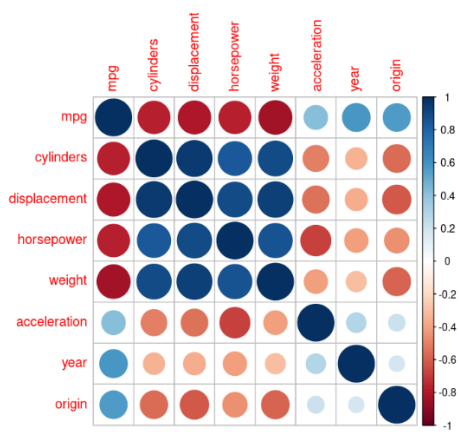
Fig. 1: Scatterplot matrix for Auto dataset

From the scatterplot matrix, we can do a screening of the variable's relationship between them. As we could see, there are many of them that shows a linear relationship (e.g., displacement ~ horsepower), and some others a polynomial relationship (mpg ~ horsepower).

b) Matrix of correlations (without variable “name”)

```
Auto_sub <- subset(Auto, select = -name)
names(Auto_sub)
Auto_cor <- cor(Auto_sub , use="complete.obs" , method="pearson")
Auto_cor
corrplot(Auto_cor)
```

```
> Auto_sub <- subset(Auto, select=-name)
> names(Auto_sub)
[1] "mpg"          "cylinders"    "displacement" "horsepower"   "weight"       "acceleration"
[7] "year"         "origin"
> Auto_cor <- cor(Auto_sub , use="complete.obs" , method="pearson")
> Auto_cor
                   mpg  cylinders displacement horsepower     weight acceleration       year     origin
mpg          1.0000000 -0.7776175   -0.8051269 -0.7784268 -0.8322442    0.4233285  0.5805410  0.5652088
cylinders   -0.7776175  1.0000000    0.9508233  0.8429834  0.8975273   -0.5046834 -0.3456474 -0.5689316
displacement -0.8051269  0.9508233    1.0000000  0.8972570  0.9329944   -0.5438005 -0.3698552 -0.6145351
horsepower  -0.7784268  0.8429834    0.8972570  1.0000000  0.8645377   -0.6891955 -0.4163615 -0.4551715
weight      -0.8322442  0.8975273    0.9329944  0.8645377  1.0000000   -0.4168392 -0.3091199 -0.5850054
acceleration  0.4233285 -0.5046834   -0.5438005 -0.6891955 -0.4168392    1.0000000  0.2903161  0.2127458
year         0.5805410 -0.3456474   -0.3698552 -0.4163615 -0.3091199    0.2903161  1.0000000  0.1815277
origin       0.5652088 -0.5689316   -0.6145351 -0.4551715 -0.5850054    0.2127458  0.1815277  1.0000000
> corrplot(Auto_cor)
```



Correlation plot for Auto_sub

We can see from above results that variables: cylinders, displacement, horsepower and weight, are highly correlated with mpg and between them (blue dots show positive correlation, red dots negative correlation). Returning to our example from scatterplot matrix, we could confirm that variables *displacement ~ horsepower* are highly correlated (0.9).

c) Multiple linear regression

```
lm.fit <- lm(data=Auto_sub , mpg ~. )
summary(lm.fit)
```

```
> lm.fit <- lm(data=Auto_sub , mpg ~. )
> summary(lm.fit)

Call:
lm(formula = mpg ~ ., data = Auto_sub)

Residuals:
    Min      1Q  Median      3Q     Max 
-9.5903 -2.1565 -0.1169  1.8690 13.0604 

Coefficients:
               Estimate Std. Error t value Pr(>|t|)    
(Intercept)  -17.218435   4.644294  -3.707  0.00024 ***
cylinders     -0.493376   0.323282  -1.526  0.12780    
displacement   0.019896   0.007515   2.647  0.00844 ** 
horsepower    -0.016951   0.013787  -1.230  0.21963    
weight        -0.006474   0.000652  -9.929  < 2e-16 ***
acceleration   0.080576   0.098845   0.815  0.41548    
year           0.750773   0.050973  14.729  < 2e-16 ***
origin         1.426141   0.278136   5.127 4.67e-07 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 3.328 on 384 degrees of freedom
Multiple R-squared:  0.8215,	Adjusted R-squared:  0.8182 
F-statistic: 252.4 on 7 and 384 DF,  p-value: < 2.2e-16
```

i. <u>Yes, there is a relationship between predictors and response</u>. We can see from above results that R^2 is 0.8215 (strong linear relationship), which means that 82.15% of the variance in “mpg” is explained by the predictors in this linear model fit. Also, the p-value is very small (close to zero) so we can conclude that there is a linear relationship between response and these predictors.

ii. Predictors that have statistically significant relationship (5% significance level) on the response:
   - Displacement
   - Weight
   - Year
   - Origin

iii. Coefficient of variable “year” (0.75) suggests that this predictor adds a significant positive weight to the expected response variable. Which means that in terms of this variable we can expect a higher mpg for higher values of the year.

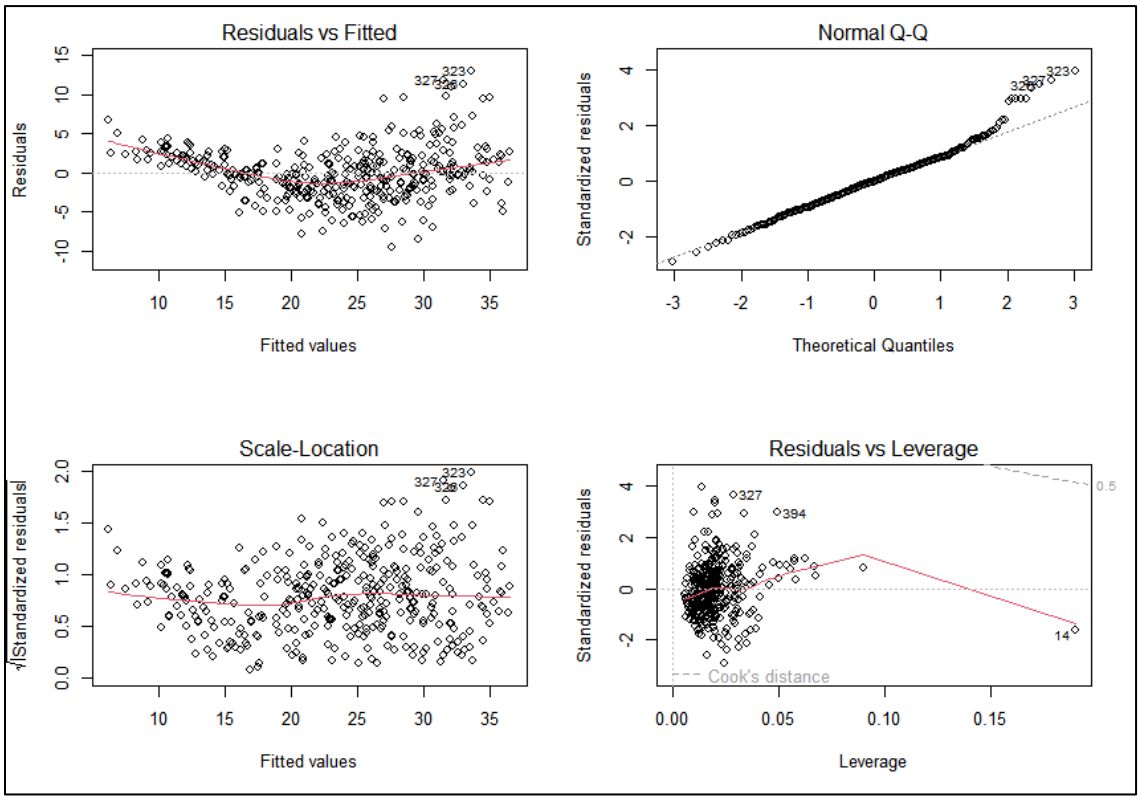
d) Diagnostic plots

Residual plots seem to be overall good in terms of linear regression assumptions, but there are some additional considerations/comments:

- <u>Linear function</u>, so expected value (mean) of residuals is zero: from residuals vs fitted values plot we can see that residuals lay randomly around residual zero line. There are some outliers which misbehave in this sense (326,327,333) but it is not that relevant.
- <u>Errors are independent</u> (i.e., cov=0): there are no visible trends on the residuals vs fitted values plot.

- Errors follow a normal distribution: Normal probability plot of the residuals shows that most of the residuals lies within the standard normal line. However, we can see there are some outliers which are not part of the same normal distribution (points 326, 327, 333; it could have been some source of noise).
- Errors have a constant variance: in the residuals scale-location plot we can see that there are no significant variations within residuals for the fitted values. However, it seems to be a higher variance around the outliers point, which is expected.

The Residuals vs Leverage Plot identifies an observation with unusually high leverage, which we can see as observation 14 in the plot.



Residual Plots for Linear Regression model (lm.fit)

Chapter 6 – Problem 8

a) Generate vectors x and e, length 100 (using set.seed() to keep same generated vectors)

```
set.seed(1)
x <- rnorm(100)
e <- rnorm(100)
```

```
> set.seed(1)
> x <- rnorm(100)
> e <- rnorm(100)
```

b) Generate response vector Y, length 100

```
B <- c(2,-1.5,0.8,1.2)
Xp_3 <- cbind(1,x,x^2,x^3)
Y <- Xp_3 %*% B + e
summary(Y)
```

```
> B <- c(2,-1.5,0.8,1.2)
> Xp_3 <- cbind(1,x,x^2,x^3)
> Y <- Xp_3 %*% B + e
> summary(Y)
       V1
 Min.   :-4.440
 1st Qu.: 1.643
 Median : 2.275
 Mean   : 2.700
 3rd Qu.: 3.310
 Max.   :20.059
```

c) Best subset selection, predictors: X, …, X^10

```
#Dataframe and best subset selection fit
df <- data.frame(x=x , y=Y)
head(df)
library(leaps)
regfit <- regsubsets(y ~ poly(x,10,raw=TRUE), data=df , nvmax=10)
reg.summary <- summary(regfit)
reg.summary
```

```
> library(leaps)
> regfit <- regsubsets(y ~ poly(x,10,raw=TRUE), data=df , nvmax = 10)
> reg.summary <- summary(regfit)
> reg.summary
```

```
> reg.summary
Subset selection object
Call: regsubsets.formula(y ~ poly(x, 10, raw = TRUE), data = df, nvmax = 10)
10 variables  (and intercept)
                         Forced in Forced out
poly(x, 10, raw = TRUE)1      FALSE      FALSE
poly(x, 10, raw = TRUE)2      FALSE      FALSE
poly(x, 10, raw = TRUE)3      FALSE      FALSE
poly(x, 10, raw = TRUE)4      FALSE      FALSE
poly(x, 10, raw = TRUE)5      FALSE      FALSE
poly(x, 10, raw = TRUE)6      FALSE      FALSE
poly(x, 10, raw = TRUE)7      FALSE      FALSE
poly(x, 10, raw = TRUE)8      FALSE      FALSE
poly(x, 10, raw = TRUE)9      FALSE      FALSE
poly(x, 10, raw = TRUE)10     FALSE      FALSE
1 subsets of each size up to 10
Selection Algorithm: exhaustive
         poly(x, 10, raw = TRUE)1 poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)3 poly(x, 10, raw = TRUE)4 poly(x, 10, raw = TRUE)5
1  ( 1 )  " "                      " "                      " "                      " "                      "*"
2  ( 1 )  " "                      "*"                      " "                      " "                      "*"
3  ( 1 )  "*"                      "*"                      "*"                      " "                      " "
4  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
5  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
6  ( 1 )  "*"                      "*"                      "*"                      " "                      " "
7  ( 1 )  "*"                      " "                      "*"                      "*"                      " "
8  ( 1 )  "*"                      " "                      " "                      "*"                      "*"
9  ( 1 )  "*"                      "*"                      " "                      "*"                      "*"
10  ( 1 ) "*"                      "*"                      "*"                      "*"                      "*"
         poly(x, 10, raw = TRUE)6 poly(x, 10, raw = TRUE)7 poly(x, 10, raw = TRUE)8 poly(x, 10, raw = TRUE)9 poly(x, 10, raw = TRUE)10
1  ( 1 )  " "                      " "                      " "                      " "                      " "
2  ( 1 )  " "                      " "                      " "                      " "                      " "
3  ( 1 )  " "                      " "                      " "                      " "                      " "
4  ( 1 )  " "                      " "                      " "                      " "                      " "
5  ( 1 )  "*"                      " "                      " "                      " "                      " "
6  ( 1 )  " "                      "*"                      "*"                      "*"                      " "
7  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
8  ( 1 )  "*"                      "*"                      "*"                      "*"                      "*"
9  ( 1 )  "*"                      "*"                      "*"                      "*"                      "*"
10  ( 1 ) "*"                      "*"                      "*"                      "*"                      "*"
```

```
#Cp, BIC, AdjR^2
names(reg.summary)
reg.summary$cp
which.min(reg.summary$cp)
reg.summary$bic
which.min(reg.summary$bic)
reg.summary$adjr2
which.max(reg.summary$adjr2)
```

```
> names(reg.summary)
[1] "which"  "rsq"    "rss"    "adjr2"  "cp"     "bic"    "outmat" "obj"
> reg.summary$cp
[1] 60.3193659 11.9260459  2.1859433  0.6067483  2.1782005  3.9955812  5.2024702  7.1106264
> which.min(reg.summary$cp)
[1] 4
> reg.summary$bic
[1] -151.5927 -185.9036 -193.0454 -192.3145 -188.1834 -183.7809 -180.0610 -175.5588
> which.min(reg.summary$bic)
[1] 3
> reg.summary$adjr2
[1] 0.7976747 0.8614857 0.8755547 0.8790237 0.8783150 0.8772556 0.8770149 0.8757914
> which.max(reg.summary$adjr2)
[1] 4
```

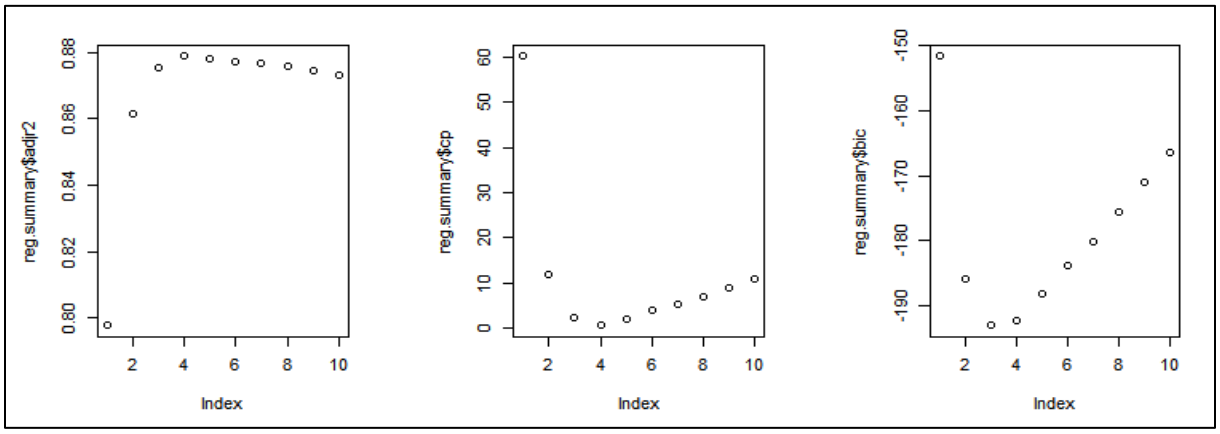
Based on these 3 indicators, best model corresponds to:

- Cp: min Cp is gotten from model 4 (using predictors: x , x^2 , x^3 , x^5)
- BIC: min BIC is gotten from model 3 (predictors: x , x^2 , x^3)
- AdjR^2: max AdjR^2 is gotten from model 4 (using predictors: x , x^2 , x^3 , x^5)

These results will be also showed and confirmed in the next plots.

```
#Plots
par(mfrow=c(3,1))
plot(reg.summary$adjr2)
plot(reg.summary$cp)
plot(reg.summary$bic)
```

```
> par(mfrow=c(1,3))
> plot(reg.summary$adjr2)
> plot(reg.summary$cp)
> plot(reg.summary$bic)
```



From left to right, as we saw from previous results, max AdjR^2 is on model no. 4, min Cp also from model no. 4, while min BIC corresponds to model no. 3.

Coefficients of the best model (in this case, it was selected model no. 4):

```
> coef(regfit,4)
(Intercept) poly(x, 10, raw = TRUE)1 poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)3 poly(x, 10, raw = TRUE)5
 2.07200775             -1.11254404              0.64575641              0.75797426              0.08072292
```

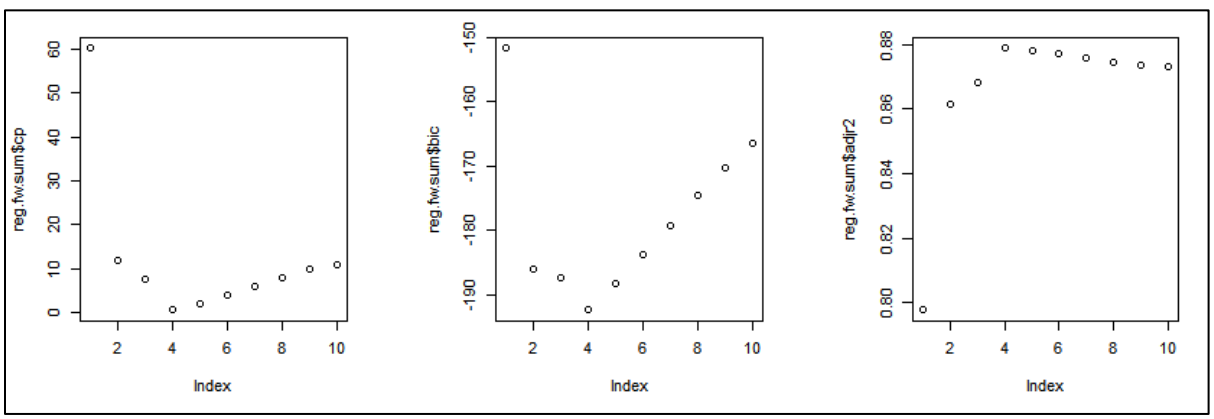
$$\hat{Y} = 2.07 - 1.11x + 0.64x^2 + 0.76x^3 + 0.08x^5 + e$$

d) Forward and backward stepwise selection, predictors: X, …, X^10

```
#Forward selection

regfit.fw <- regsubsets(y ~ poly(x,10,raw=TRUE), data=df,
method="forward" , nvmax=10)
reg.fw.sum <- summary(regfit.fw
reg.fw.sum
which.min(reg.fw.sum$cp)
which.min(reg.fw.sum$bic)
which.max(reg.fw.sum$adjr2)
par(mfrow=c(1,3))
plot(reg.fw.sum$cp)
plot(reg.fw.sum$bic)
plot(reg.fw.sum$adjr2)

#Backward selection

regfit.bw <- regsubsets(y ~ poly(x,10,raw=TRUE), data=df,
method="backward" , nvmax=10)
reg.bw.sum <- summary(regfit.bw)
reg.bw.sum
which.min(reg.bw.sum$cp)
which.min(reg.bw.sum$bic)
which.max(reg.bw.sum$adjr2)
par(mfrow=c(1,3))
plot(reg.bw.sum$cp)
```

```
plot(reg.bw.sum$bic)
plot(reg.bw.sum$adjr2)
```

```
> regfit.fw <- regsubsets(y ~ poly(x,10,raw=TRUE), data=df,  method="forward" , nvmax = 10)
> reg.fw.sum <- summary(regfit.fw)
> reg.fw.sum
Subset selection object
Call: regsubsets.formula(y ~ poly(x, 10, raw = TRUE), data = df, method = "forward",
    nvmax = 10)
10 variables  (and intercept)
                          Forced in Forced out
poly(x, 10, raw = TRUE)1      FALSE      FALSE
poly(x, 10, raw = TRUE)2      FALSE      FALSE
poly(x, 10, raw = TRUE)3      FALSE      FALSE
poly(x, 10, raw = TRUE)4      FALSE      FALSE
poly(x, 10, raw = TRUE)5      FALSE      FALSE
poly(x, 10, raw = TRUE)6      FALSE      FALSE
poly(x, 10, raw = TRUE)7      FALSE      FALSE
poly(x, 10, raw = TRUE)8      FALSE      FALSE
poly(x, 10, raw = TRUE)9      FALSE      FALSE
poly(x, 10, raw = TRUE)10     FALSE      FALSE
1 subsets of each size up to 10
Selection Algorithm: forward
          poly(x, 10, raw = TRUE)1 poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)3 poly(x, 10, raw = TRUE)4 poly(x, 10, raw = TRUE)5
1  ( 1 )  " "                      " "                      " "                      " "                      "*"
2  ( 1 )  " "                      "*"                      " "                      " "                      "*"
3  ( 1 )  "*"                      "*"                      " "                      " "                      "*"
4  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
5  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
6  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
7  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
8  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
9  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
10  ( 1 ) "*"                      "*"                      "*"                      "*"                      "*"
          poly(x, 10, raw = TRUE)6 poly(x, 10, raw = TRUE)7 poly(x, 10, raw = TRUE)8 poly(x, 10, raw = TRUE)9 poly(x, 10, raw = TRUE)10
1  ( 1 )  " "                      " "                      " "                      " "                      " "
2  ( 1 )  " "                      " "                      " "                      " "                      " "
3  ( 1 )  " "                      " "                      " "                      " "                      " "
4  ( 1 )  " "                      " "                      " "                      " "                      " "
5  ( 1 )  "*"                      " "                      " "                      " "                      " "
6  ( 1 )  "*"                      " "                      " "                      "*"                      " "
7  ( 1 )  "*"                      "*"                      " "                      "*"                      " "
8  ( 1 )  "*"                      "*"                      "*"                      "*"                      " "
9  ( 1 )  "*"                      "*"                      "*"                      "*"                      "*"
10  ( 1 ) "*"                      "*"                      "*"                      "*"                      "*"
> which.min(reg.fw.sum$cp)
[1] 4
> which.min(reg.fw.sum$bic)
[1] 4
> which.max(reg.fw.sum$adjr2)
[1] 4
> par(mfrow=c(1,3))
> plot(reg.fw.sum$cp)
> plot(reg.fw.sum$bic)
> plot(reg.fw.sum$adjr2)
```



Using forward stepwise selection, we came up with the following results:

- Cp: min Cp is gotten from model 4 (using predictors: x , x^2 , x^3 , x^5)
- BIC: min BIC is gotten from model 4 (predictors: x , x^2 , x^3, x^5)
- AdjR^2: max AdjR^2 is gotten from model 4 (using predictors: x , x^2 , x^3 , x^5)

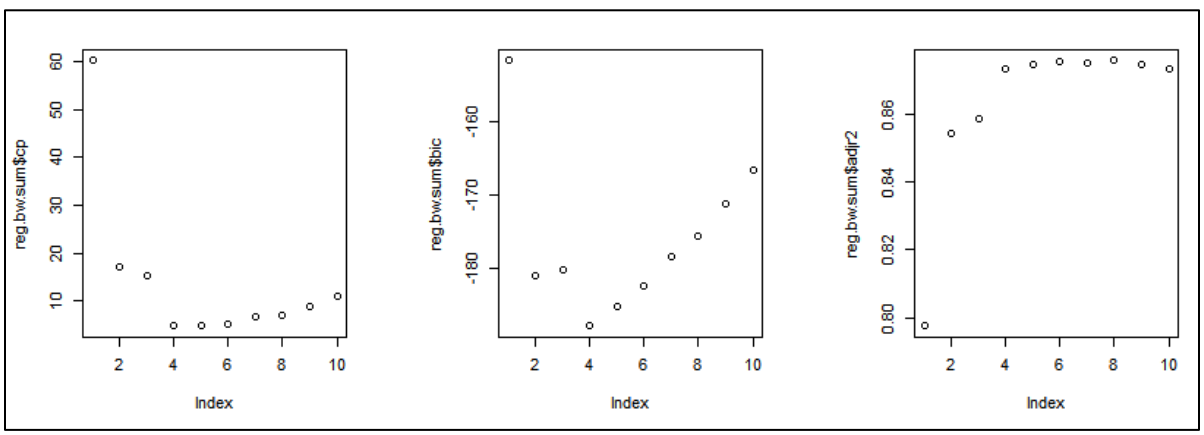
So, it turns out that based on these three different indicators, the best model is the same for each case which is model no. 4.

Coefficients of model 4 using forward stepwise selection

```
> coef(regfit.fw,4)
             (Intercept) poly(x, 10, raw = TRUE)1 poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)3 poly(x, 10, raw = TRUE)5
              2.07200775              -1.11254404               0.64575641               0.75797426               0.08072292
```

$$\hat{Y} = 2.07 - 1.11x + 0.65x^2 + 0.76x^3 + 0.08x^5 + e$$

```
> regfit.bw <- regsubsets(y ~ poly(x,10,raw=TRUE), data=df,  method="backward" , nvmax = 10)
> reg.bw.sum <- summary(regfit.bw)
> reg.bw.sum
Subset selection object
Call: regsubsets.formula(y ~ poly(x, 10, raw = TRUE), data = df, method = "backward",
    nvmax = 10)
10 variables  (and intercept)
                          Forced in Forced out
poly(x, 10, raw = TRUE)1      FALSE      FALSE
poly(x, 10, raw = TRUE)2      FALSE      FALSE
poly(x, 10, raw = TRUE)3      FALSE      FALSE
poly(x, 10, raw = TRUE)4      FALSE      FALSE
poly(x, 10, raw = TRUE)5      FALSE      FALSE
poly(x, 10, raw = TRUE)6      FALSE      FALSE
poly(x, 10, raw = TRUE)7      FALSE      FALSE
poly(x, 10, raw = TRUE)8      FALSE      FALSE
poly(x, 10, raw = TRUE)9      FALSE      FALSE
poly(x, 10, raw = TRUE)10     FALSE      FALSE
1 subsets of each size up to 10
Selection Algorithm: backward
         poly(x, 10, raw = TRUE)1 poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)3 poly(x, 10, raw = TRUE)4 poly(x, 10, raw = TRUE)5
1  ( 1 )  " "                      " "                      " "                      " "                      "*"
2  ( 1 )  " "                      " "                      " "                      "*"                      "*"
3  ( 1 )  "*"                      " "                      " "                      "*"                      "*"
4  ( 1 )  "*"                      " "                      " "                      "*"                      "*"
5  ( 1 )  "*"                      " "                      " "                      "*"                      "*"
6  ( 1 )  "*"                      " "                      " "                      "*"                      "*"
7  ( 1 )  "*"                      " "                      " "                      "*"                      "*"
8  ( 1 )  "*"                      " "                      " "                      "*"                      "*"
9  ( 1 )  "*"                      "*"                      " "                      "*"                      "*"
10  ( 1 ) "*"                      "*"                      "*"                      "*"                      "*"
         poly(x, 10, raw = TRUE)6 poly(x, 10, raw = TRUE)7 poly(x, 10, raw = TRUE)8 poly(x, 10, raw = TRUE)9 poly(x, 10, raw = TRUE)10
1  ( 1 )  " "                      " "                      " "                      " "                      " "
2  ( 1 )  " "                      " "                      " "                      " "                      " "
3  ( 1 )  " "                      " "                      " "                      " "                      " "
4  ( 1 )  " "                      "*"                      " "                      " "                      " "
5  ( 1 )  "*"                      "*"                      " "                      " "                      " "
6  ( 1 )  "*"                      "*"                      "*"                      " "                      " "
7  ( 1 )  "*"                      "*"                      "*"                      " "                      "*"
8  ( 1 )  "*"                      "*"                      "*"                      "*"                      "*"
9  ( 1 )  "*"                      "*"                      "*"                      "*"                      "*"
10  ( 1 ) "*"                      "*"                      "*"                      "*"                      "*"
> which.min(reg.bw.sum$cp)
[1] 4
> which.min(reg.bw.sum$bic)
[1] 4
> which.max(reg.bw.sum$adjr2)
[1] 8
> par(mfrow=c(1,3))
> plot(reg.bw.sum$cp)
> plot(reg.bw.sum$bic)
> plot(reg.bw.sum$adjr2)
```



Using backward stepwise selection, we came up with the following results:

- Cp: min Cp is gotten from model 4 (using predictors: x , x^2 , x^3 , x^5)

- BIC: min BIC is gotten from model 4 (predictors: x , x^2 , x^3, x^5)
- AdjR^2: max AdjR^2 is gotten from model 8 (using predictors: x , x^2 , x^3 , x^5)

So we obtained same results as using forward stepwise selection in terms of Cp and BIC. However, based on AdjR^2 turns out that best model is no. 8.

Coefficients for models 4 and 8

```
> coef(regfit.fw,4)
            (Intercept) poly(x, 10, raw = TRUE)1 poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)3 poly(x, 10, raw = TRUE)5
             2.07200775             -1.11254404              0.64575641              0.75797426              0.08072292
> coef(regfit.fw,8)
            (Intercept) poly(x, 10, raw = TRUE)1 poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)3 poly(x, 10, raw = TRUE)5
            2.102542912            -1.188736231             0.589363630             1.050608304            -0.181106683
poly(x, 10, raw = TRUE)6 poly(x, 10, raw = TRUE)7 poly(x, 10, raw = TRUE)8 poly(x, 10, raw = TRUE)9
            -0.009513473             0.082957234             0.003073173            -0.008429573
```

Model 4: $\hat{Y} = 2.07 - 1.11x + 0.65x^2 + 0.76x^3 + 0.08x^5 + e$

Model 8: $\hat{Y} = 2.10 - 1.19x + 0.59x^2 + 1.06x^3 - 0.18x^5 - 0.01x^6 + 0.08x^7 + 0.003\,x^8 - 0.008x^9 + e$

Summarizing all these results in terms of best model, we have the following:

	Min Cp	Min BIC	Max AdjR^2
Best subset	4	3	4
Forward stepwise	4	4	4
Backward stepwise	4	4	8

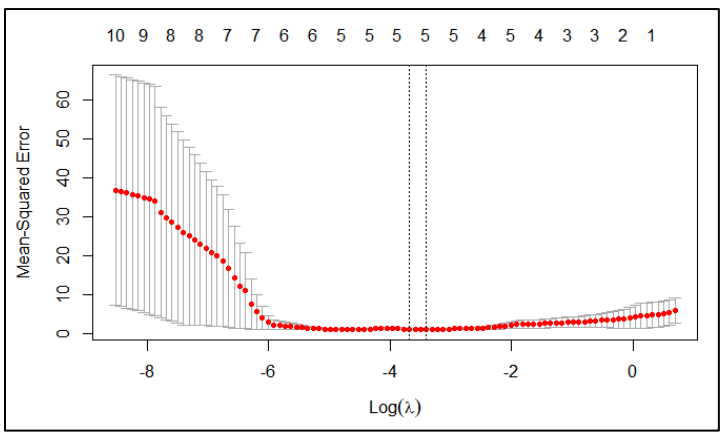
Somehow, we could say that overall model 4 performs best in most of the methods and indicators.

$$\hat{Y} = 2.07 - 1.11x + 0.65x^2 + 0.76x^3 + 0.08x^5 + e$$

e) Lasso model (cross-validation to choose “lambda”), predictors: X, …, X^10.

```
library(glmnet)
X_reg <- model.matrix(y ~ poly(x,10,raw=TRUE), data = df)[,-1]
set.seed(1)
train <- sample(1:nrow(X_reg) , nrow(X_reg)/2 )
test <- (-train)
y.test <- Y[test]
cv.lasso <- cv.glmnet( X_reg[train,] , Y[train] , alpha = 1)
plot(cv.lasso)
bestlam <- cv.lasso$lambda.min
bestlam
lasso.mod <- glmnet( X_reg , Y , alpha=1 , lambda=bestlam)
coef(lasso.mod)
```

```
> library(glmnet)
> X_reg <- model.matrix(y ~ poly(x,10,raw=TRUE), data = df)[,-1]
> set.seed(1)
> train <- sample(1:nrow(X_reg) , nrow(X_reg)/2 )
> test <- (-train)
> y.test <- Y[test]
> cv.lasso <- cv.glmnet( X_reg[train,] , Y[train] , alpha = 1)
> plot(cv.lasso)
> bestlam <- cv.lasso$lambda.min
> bestlam
[1] 0.02517312
> lasso.mod <- glmnet( X_reg , Y , alpha=1 , lambda=bestlam)
> coef(lasso.mod)
11 x 1 sparse Matrix of class "dgCMatrix"
                                  s0
(Intercept)                2.11574879
poly(x, 10, raw = TRUE)1  -0.78711668
poly(x, 10, raw = TRUE)2   0.51304166
poly(x, 10, raw = TRUE)3   0.52533134
poly(x, 10, raw = TRUE)4   0.02689683
poly(x, 10, raw = TRUE)5   0.11107916
poly(x, 10, raw = TRUE)6   .
poly(x, 10, raw = TRUE)7   .
poly(x, 10, raw = TRUE)8   .
poly(x, 10, raw = TRUE)9   .
poly(x, 10, raw = TRUE)10  .
```



Using cross-validation to select optimal value of lambda, this lambda is 0.0251.

Applying lasso regression to fit a model, the best model using this value of lambda uses 5 predictors. The fitted model is the following:

$$\hat{Y} = 2.12 - 0.78x + 0.51x^2 + 0.53x^3 + +0.03x^4 + 1.11x^5 + e$$

f) Generate response vector Y_7, perform Best subset and Lasso

```
##bss fit
Y_7 <- 5 + 2*(x^7) + e
df_7 <- data.frame( Y_7 = Y_7 , x=x)
bss.fit_7 <- regsubsets(Y_7 ~ poly(x,10,raw=TRUE) , data = df_7)
bss.fit_7_sum <- summary(bss.fit_7)
bss.fit_7_sum
which.min(bss.fit_7_sum$cp)
which.min(bss.fit_7_sum$bic)
which.max(bss.fit_7_sum$adjr2)
```

```
> Y_7 <- 5 + 2*(x^7) + e
> df_7 <- data.frame( Y_7 = Y_7 , x=x)
> bss.fit_7 <- regsubsets(Y_7 ~ poly(x,10,raw=TRUE) , data = df_7)
> bss.fit_7_sum <- summary(bss.fit_7)
> bss.fit_7_sum
Subset selection object
Call: regsubsets.formula(Y_7 ~ poly(x, 10, raw = TRUE), data = df_7)
10 Variables  (and intercept)
                          Forced in Forced out
poly(x, 10, raw = TRUE)1      FALSE      FALSE
poly(x, 10, raw = TRUE)2      FALSE      FALSE
poly(x, 10, raw = TRUE)3      FALSE      FALSE
poly(x, 10, raw = TRUE)4      FALSE      FALSE
poly(x, 10, raw = TRUE)5      FALSE      FALSE
poly(x, 10, raw = TRUE)6      FALSE      FALSE
poly(x, 10, raw = TRUE)7      FALSE      FALSE
poly(x, 10, raw = TRUE)8      FALSE      FALSE
poly(x, 10, raw = TRUE)9      FALSE      FALSE
poly(x, 10, raw = TRUE)10     FALSE      FALSE
1 subsets of each size up to 8
Selection Algorithm: exhaustive
         poly(x, 10, raw = TRUE)1 poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)3 poly(x, 10, raw = TRUE)4 poly(x, 10, raw = TRUE)5
1  ( 1 ) " "                      " "                      " "                      " "                      " "
2  ( 1 ) " "                      "*"                      " "                      " "                      " "
3  ( 1 ) " "                      "*"                      " "                      " "                      "*"
4  ( 1 ) "*"                      "*"                      "*"                      " "                      " "
5  ( 1 ) "*"                      "*"                      "*"                      "*"                      " "
6  ( 1 ) "*"                      " "                      "*"                      " "                      " "
7  ( 1 ) "*"                      " "                      "*"                      " "                      "*"
8  ( 1 ) "*"                      "*"                      "*"                      "*"                      " "
         poly(x, 10, raw = TRUE)6 poly(x, 10, raw = TRUE)7 poly(x, 10, raw = TRUE)8 poly(x, 10, raw = TRUE)9 poly(x, 10, raw = TRUE)10
1  ( 1 ) " "                      "*"                      " "                      " "                      " "
2  ( 1 ) " "                      "*"                      " "                      " "                      " "
3  ( 1 ) " "                      "*"                      " "                      " "                      " "
4  ( 1 ) " "                      "*"                      " "                      " "                      " "
5  ( 1 ) " "                      "*"                      " "                      " "                      " "
6  ( 1 ) "*"                      "*"                      "*"                      " "                      "*"
7  ( 1 ) "*"                      "*"                      "*"                      " "                      "*"
8  ( 1 ) "*"                      "*"                      "*"                      " "                      "*"
> which.min(bss.fit_7_sum$cp)
[1] 2
> which.min(bss.fit_7_sum$bic)
[1] 1
> which.max(bss.fit_7_sum$adjr2)
[1] 4
> coef(bss.fit_7,2)
             (Intercept) poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)7
               5.0704904               -0.1417084                2.0015552
> coef(bss.fit_7,1)
             (Intercept) poly(x, 10, raw = TRUE)7
                4.95894                  2.00077
> coef(bss.fit_7,4)
             (Intercept) poly(x, 10, raw = TRUE)1 poly(x, 10, raw = TRUE)2 poly(x, 10, raw = TRUE)3 poly(x, 10, raw = TRUE)7
               5.0762524                0.2914016               -0.1617671               -0.2526527                2.0091338
```

```
##Lasso fit

cv.lasso_7 <- cv.glmnet( X_reg[train,] , Y_7[train] , alpha = 1)
plot(cv.lasso_7)
bestlam_7 <- cv.lasso_7$lambda.min
bestlam_7
lasso.mod_7 <- glmnet( X_reg , Y_7 , alpha=1 , lambda=bestlam_7)
coef(lasso.mod_7)
```

```
> cv.lasso_7 <- cv.glmnet( X_reg[train,] , Y_7[train] , alpha = 1)
> plot(cv.lasso_7)
> bestlam_7 <- cv.lasso_7$lambda.min
> bestlam_7
[1] 3.042966
> lasso.mod_7 <- glmnet( X_reg , Y_7 , alpha=1 , lambda=bestlam_7)
> coef(lasso.mod_7)
11 x 1 sparse Matrix of class "dgCMatrix"
                                  s0
(Intercept)               5.169291526
poly(x, 10, raw = TRUE)1  .
poly(x, 10, raw = TRUE)2  .
poly(x, 10, raw = TRUE)3  .
poly(x, 10, raw = TRUE)4  .
poly(x, 10, raw = TRUE)5  .
poly(x, 10, raw = TRUE)6  .
poly(x, 10, raw = TRUE)7  1.938459188
poly(x, 10, raw = TRUE)8  .
poly(x, 10, raw = TRUE)9  0.002214598
poly(x, 10, raw = TRUE)10 .
```

As we can see, all of the models obtained as the best model based on different indicators includes the predictor x^7 as one of the relevant predictors. This makes sense as the response vector Y_7 was generated from vector x^7.

Chapter 6 – Problem 9 (a,b,c,d,g)

a) Split training set and testing set

```
College <- read.table("College.data" , header = TRUE)
View(College)
set.seed(2)
train2 <- sample(1:nrow(College) , nrow(College)/3*2 )
test2 <- (-train2)
df.train2 <- College[train2,]
df.test2 <- College [test2,]
```

```
> set.seed(2)
> train2 <- sample(1:nrow(College) , nrow(College)/3*2 )
> test2 <- (-train2)
> df.train2 <- College[train2,]
> df.test2 <- College [test2,]
```

b) Linear model (least squares)

```
X2_reg <- model.matrix(Apps ~ . , data=College)[,-1]
y.train2 <- df.train2$Apps
y.test2 <- df.test2$Apps
#LeastSquare fit
lm.fit2 <- lm(Apps ~ . , data=df.train2)
summary(lm.fit2)
lm.pred2 <- predict(lm.fit2 , newdata = df.test2)
lm.error <- mean((lm.pred2 - y.test2)^2)
lm.error
```

```
> X2_reg <- model.matrix(Apps ~ . , data=College)[,-1]
> y.train2 <- df.train2$Apps
> y.test2 <- df.test2$Apps
> #LeastSquare fit
> lm.fit2 <- lm(Apps ~ . , data=df.train2)
```

```
> summary(lm.fit2)

Call:
lm(formula = Apps ~ ., data = df.train2)

Residuals:
    Min      1Q  Median      3Q     Max 
-5028.3  -424.1    -8.0   304.9  7204.9 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept) -668.06818  512.05099  -1.305  0.19260    
PrivateYes  -473.74099  169.20585  -2.800  0.00531 ** 
Accept         1.62920    0.04498  36.218  < 2e-16 ***
Enroll        -1.16615    0.21743  -5.363 1.25e-07 ***
Top10perc     51.58736    6.96230   7.410 5.44e-13 ***
Top25perc    -16.77020    5.73568  -2.924  0.00361 ** 
F.Undergrad    0.09432    0.03788   2.490  0.01309 *  
P.Undergrad    0.05700    0.03598   1.584  0.11373    
Outstate      -0.07856    0.02430  -3.233  0.00130 ** 
Room.Board     0.18594    0.06000   3.099  0.00205 ** 
Books          0.12853    0.32212   0.399  0.69006    
Personal       0.05740    0.07326   0.784  0.43367    
PhD           -6.72818    5.74465  -1.171  0.24207    
Terminal      -6.22921    6.41427  -0.971  0.33194    
S.F.Ratio     26.85038   17.67576   1.519  0.12938    
perc.alumni    5.06688    5.13239   0.987  0.32400    
Expend         0.07321    0.01666   4.396 1.35e-05 ***
Grad.Rate      6.16562    3.60564   1.710  0.08789 .  
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 1060 on 500 degrees of freedom
Multiple R-squared:  0.9398,	Adjusted R-squared:  0.9377 
F-statistic: 459.1 on 17 and 500 DF,  p-value: < 2.2e-16

> lm.pred2 <- predict(lm.fit2 , newdata = df.test2)
> lm.error <- mean((lm.pred2 - y.test2)^2)
> lm.error
[1] 1039755
```

c) Ridge regression (cross-validation lambda)

```
cv.ridge2 <- cv.glmnet( X2_reg[train2,] , y.train2 , alpha=0 )
plot(cv.ridge2)
bestlam2.ridge <- cv.ridge2$lambda.min
bestlam2.ridge
ridge.mod2 <- glmnet( X2_reg[train2,] , y.train2 , alpha=0 ,
lambda=bestlam2.ridge)
ridge.pred2 <- predict( ridge.mod2 , newx = X2_reg[test2,] )
ridge.error <- mean( (ridge.pred2 - y.test2)^2 )
ridge.error
```

```
> cv.ridge2 <- cv.glmnet( X2_reg[train2,] , y.train2 , alpha=0 )
> plot(cv.ridge2)
> bestlam2.ridge <- cv.ridge2$lambda.min
> bestlam2.ridge
[1] 404.6874
> ridge.mod2 <- glmnet( X2_reg[train2,] , y.train2 , alpha=0 , lambda=bestlam2.ridge)
> ridge.pred2 <- predict( ridge.mod2 , newx = X2_reg[test2,] )
> ridge.error <- mean( (ridge.pred2 - y.test2)^2 )
> ridge.error
[1] 1032917
```

d) Lasso regression (cross-validation lambda)

```
cv.lasso2 <- cv.glmnet( X2_reg[train2,] , y.train2 , alpha=1 )
plot(cv.lasso2)
bestlam2.lasso <- cv.lasso2$lambda.min
bestlam2.lasso
lasso.mod2 <- glmnet( X2_reg[train2,] , y.train2 , alpha=1 ,
lambda=bestlam2.lasso)
lasso.pred2 <- predict( lasso.mod2 , newx = X2_reg[test2,] )
lasso.error <- mean( (lasso.pred2 - y.test2)^2 )
lasso.error
```

```
> cv.lasso2 <- cv.glmnet( X2_reg[train2,] , y.train2 , alpha=1 )
> plot(cv.lasso2)
> bestlam2.lasso <- cv.lasso2$lambda.min
> bestlam2.lasso
[1] 2.159695
> lasso.mod2 <- glmnet( X2_reg[train2,] , y.train2 , alpha=1 , lambda=bestlam2.lasso)
> lasso.pred2 <- predict( lasso.mod2 , newx = X2_reg[test2,] )
> lasso.error <- mean( (lasso.pred2 - y.test2)^2 )
> lasso.error
[1] 1029693
```

e) When we use a least squares linear regression, we get a very good training model because we obtain a R^2 of 0.94. So, this model has a low bias; however, test error is the largest one compared to the other methods. So we have a low bias but higher variance when using this model on new data.
On the other hand, Ridge and Lasso regression performs better in terms of test error minimization, having Lasso the lowest test error. This is good as the variance is smaller and so, this model performs better on new data.

<u>Chapter 6 – Problem 10</u>

a) Data set, p=20 , n=1000

```
set.seed(3)
x_20 <- matrix(rnorm(20000) , ncol=20 , nrow=1000)
Xm <- cbind(1 , x_20)
e2 <- rnorm(1000)
B2 <- c(5 , 3.5 , -2 , 0 , 1 , 0 , 1.3 , 0 , 0 , -1 , 0.5 , 0 , -
2.5 , 0 , 1.9 , 0.3 , 0 , -0.8 , 2 , 0 , 1)
Y2 <- Xm %*%  B2 + e2
ds <- data.frame(cbind (Y2 , x_20))
```

```
> set.seed(3)
> x_20 <- matrix(rnorm(20000) , ncol=20 , nrow=1000)
> Xm <- cbind(1 , x_20)
> e2 <- rnorm(1000)
> B2 <- c(5 , 3.5 , -2 , 0 , 1 , 0 , 1.3 , 0 , 0 , -1 , 0.5 , 0 , -2.5 , 0 , 1.9 , 0.3 , 0 , -0.8 , 2 , 0 , 1)
> Y2 <- Xm %*%  B2 + e2
> ds <- data.frame(cbind (Y2 , x_20))
```

b) Training (100 obs) and testing (900 obs) sets

```
set.seed(3)
train3 <- sample(1:nrow(ds) , 100)
test3 <- (-train3)
ds.train3 <- ds[train3,]
x.train3 <- as.matrix(ds.train3[ , -1])
```

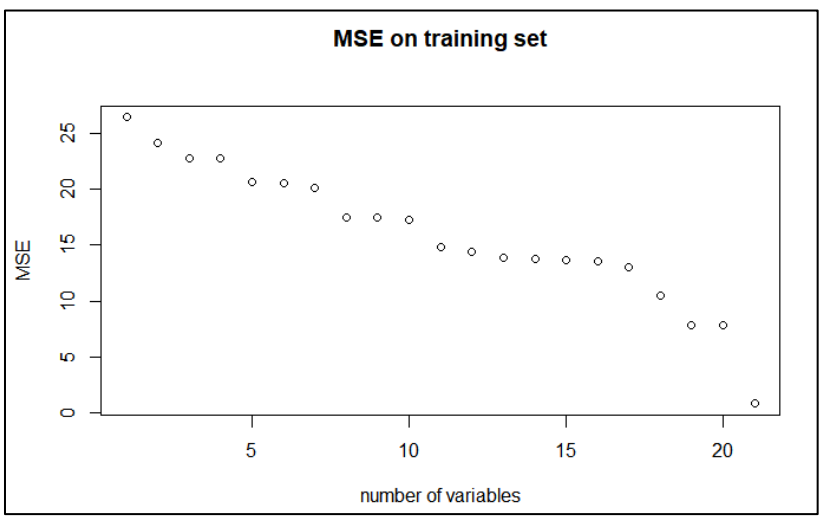
```
y.train3 <- as.matrix(ds.train3[,1])
ds.test3 <- ds[test3,]
x.test3 <- as.matrix(ds.test3[,-1])
y.test3 <- as.matrix(ds.test3[,1])
```

```
> set.seed(3)
> train3 <- sample(1:nrow(ds) , 100)
> test3 <- (-train3)
> ds.train3 <- ds[train3,]
> x.train3 <- as.matrix(ds.train3[ , -1])
> y.train3 <- as.matrix(ds.train3[,1])
> ds.test3 <- ds[test3,]
> x.test3 <- as.matrix(ds.test3[,-1])
> y.test3 <- as.matrix(ds.test3[,1])
```

c) Best subset selection

```
bss.fit <- regsubsets(Y ~ . , data = ds.train3 , nvmax=20)
bss_mse <- bss.fit$rss/(nrow(ds.train3))
plot(bss_mse , main = "MSE on training set" , xlab = "number of
variables" , ylab = "MSE")
```

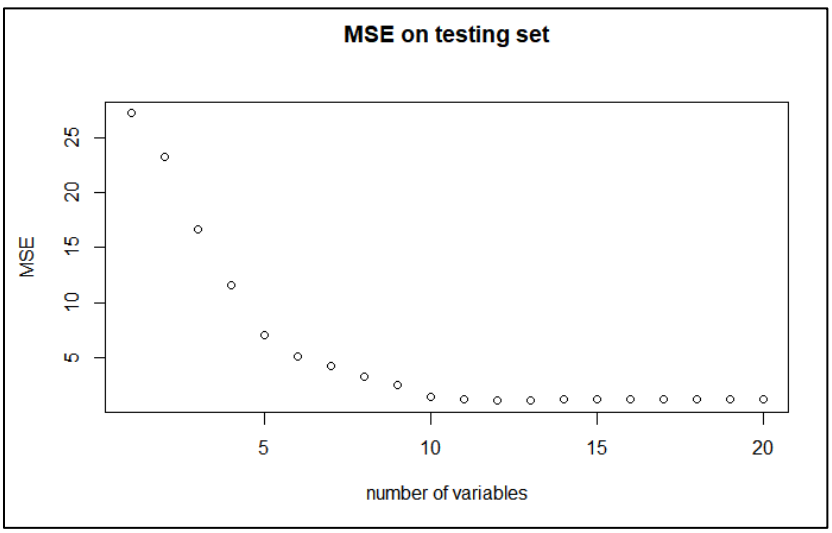
```
> bss.fit <- regsubsets(Y ~ . , data = ds.train3 , nvmax=20)
> bss_mse <- bss.fit$rss/(nrow(ds.train3))
> plot(bss_mse , main = "MSE on training set" , xlab = "number of variables" , ylab = "MSE")
```



d) Plot test MSE

```
test_errors = rep(NA,19)
test.mat <- model.matrix(Y ~ ., data = ds.test3)
for (i in 1:20){
  coefs = coef(bss.fit, id=i)
  pred = test.mat[, names(coefs)] %*% coefs
  test_errors[i] = mean((y.test3-pred)^2)
}
plot(test_errors , main = "MSE on testing set" , xlab = "number
of variables" , ylab = "MSE")
data.frame(x=1:20 , y=test_errors)
which.min(test_errors)
```

```
> test_errors = rep(NA,19)
> test.mat <- model.matrix(Y ~ ., data = ds.test3)
> for (i in 1:20){
+   coefs = coef(bss.fit, id=i)
+   pred = test.mat[, names(coefs)] %*% coefs
+   test_errors[i] = mean((y.test3-pred)^2)
+ }
> plot(test_errors)
> plot(test_errors , main = "MSE on testing set" , xlab = "number of variables" , ylab = "MSE")
> data.frame(x=1:20 , y=test_errors)
    x         y
1   1 27.172760
2   2 23.172997
3   3 16.619928
4   4 11.649756
5   5  7.131564
6   6  5.195850
7   7  4.325141
8   8  3.286581
9   9  2.593662
10 10  1.533589
11 11  1.271768
12 12  1.122579
13 13  1.187889
14 14  1.257490
15 15  1.259243
16 16  1.275162
17 17  1.289137
18 18  1.298114
19 19  1.315886
20 20  1.316895
> which.min(test_errors)
[1] 12
```



e) From above results, we can see how the training error decreases as we add more and more predictors. This results as having a low training error. But when we check the test error (error on new data, the testing set) we can appreciate that the test error decreases until the model has 12 variables, and then this starts to increase again. So, for model no. 12 we obtain the minimum test error (variance) while the model is also performing well on the training set.

```
> coef(bss.fit,12)
(Intercept)         x.1         x.2         x.4         x.6         x.9        x.10        x.12        x.14
  5.0227105   3.3852965  -1.9850103   0.9703853   1.4556985  -0.8970800   0.5917470  -2.3905866   1.8868847
       x.15        x.17        x.18        x.20
  0.3047946  -0.9450813   1.9675447   0.9683312
```

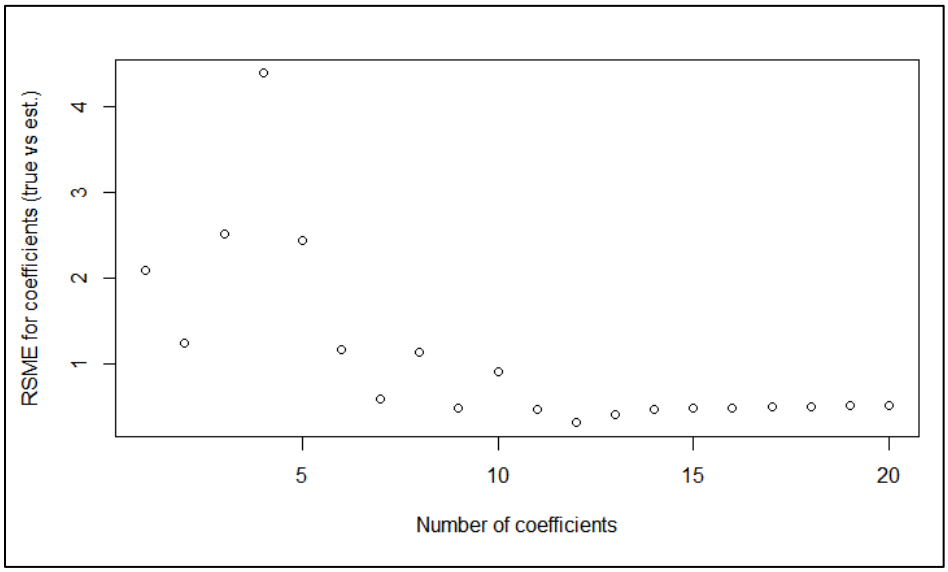
f) The true model coefficients are very close to the fitted model with lowest test error. Below we compare both vectors B2 (from the true model, the one which was used to generate the data) and the coefficients of the fitted model.
We can see that all the coefficients in B2 assigned as zero are not part of the fitted model, and the included predictors are very close to the true value. So, results are accordingly.

```
> B2
 [1]  5.0  3.5 -2.0  0.0  1.0  0.0  1.3  0.0  0.0 -1.0  0.5  0.0 -2.5  0.0  1.9  0.3  0.0 -0.8  2.0  0.0  1.0
> coef(bss.fit,12)
(Intercept)         x.1         x.2         x.4         x.6         x.9        x.10        x.12        x.14
  5.0227105   3.3852965  -1.9850103   0.9703853   1.4556985  -0.8970800   0.5917470  -2.3905866   1.8868847
       x.15        x.17        x.18        x.20
  0.3047946  -0.9450813   1.9675447   0.9683312
```

g) Plot for differences in coefficients (true vs estimated)

```
test_errors = rep(NA,19)
test.mat <- model.matrix(Y ~ ., data = ds.test3)
for (i in 1:20){
  coefs = coef(bss.fit, id=i)
  pred = test.mat[, names(coefs)] %*% coefs
  test_errors[i] = mean((y.test3-pred)^2)
}
plot(test_errors , main = "MSE on testing set" , xlab = "number
of variables" , ylab = "MSE")
data.frame(x=1:20 , y=test_errors)
which.min(test_errors)

coef_errors <- rep(NA,20)
x_20_cols <- c("(Intercept)" , colnames(x_20 , do.NULL = FALSE ,
prefix = "x."))

for (i in 1:20){
  coef = coef(bss.fit, id=i)
  coef_errors[i] = sqrt(sum((B2[x_20_cols %in% names(coef)] -
coef[names(coef) %in% x_20_cols])^2) + sum(B2[!(x_20_cols %in%
names(coef))])^2)
}
df_coef <- data.frame(no_coef = 1:20 , error_est = coef_errors)
par(mfrow=c(1,1))
plot(df_coef , xlab = "Number of coefficients" , ylab = "RSME for
coefficients (true vs est.)")
lowest_error <- which.min(coef_errors)
lowest_error
```

```
> coef_errors <- rep(NA,20)
> x_20_cols <- c("(Intercept)" , colnames(x_20 , do.NULL = FALSE , prefix = "x."))
>
> for (i in 1:20){
+   coef = coef(bss.fit, id=i)
+   coef_errors[i] = sqrt(sum((B2[x_20_cols %in% names(coef)] - coef[names(coef) %in% x_20_cols])^2) + sum(B2[!(x_20_cols %in% names(coef))])^2)
+ }
> df_coef <- data.frame(no_coef = 1:20 , error_est = coef_errors)
> par(mfrow=c(1,1))
> plot(df_coef , xlab = "Number of coefficients" , ylab = "RSME for coefficients (true vs est.)")
> lowest_error <- which.min(coef_errors)
> lowest_error
[1] 12
```



We could see from above results that the model that results with the lowest RMSE between true and estimated coefficients is the one that includes 12 predictors. Same as the results obtained from the lowest testing error. This is expected to be this way, as the response vector was generated with specifically these coefficients and the rest were set equal to zero, so this plot confirms that our regression model fit is the one that performs best.